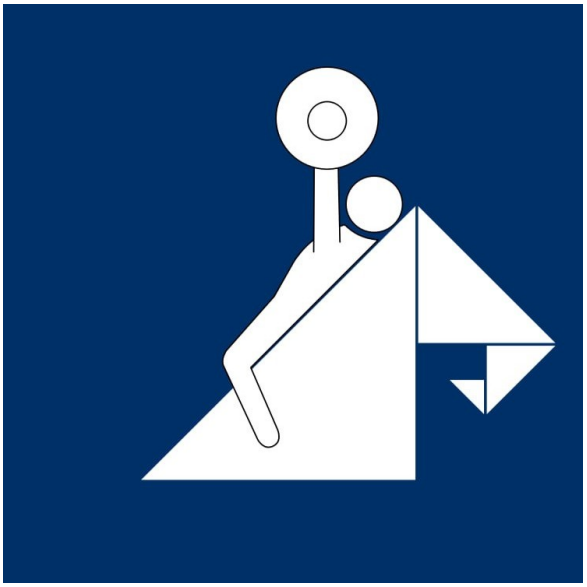
Universidad de La Habana
Facultad de Matemática y Computación
Año: 3ro
Asignatura: Ingeniería de Software



Clase Practica 4 de Arquitecturas de Software:

Nombre de Equipo: SQLote

Nombre y Apellidos	**Pregunta**
Ana Laura Oliva Avilés	3
Andriu Ferreyra Rodríguez	7
Diana Martínez Almanza	7
Yisell Velázquez Ochoa	3
José Antonio Bello Cabrera	3

Enlace al repositorio

Curso: 2026-2027

Índice

3. Pregunta 3 **2**

3.1. Inciso a) Identificación y justificación del patrón 2

3.2. Inciso b) Implementación en pseudocódigo 2

3.3. Inciso c) Documentos jerárquicos (Composite e Iterator) 3

3.4. Inciso d) Comparativa y ejecución en paralelo 4

7. Pregunta 7 **6**

7.1. Inciso a) Identificación y justificación del patrón 6

7.2. Inciso b) Implementación en pseudocódigo 6

7.3. Inciso c) Incorporación de nuevas actividades sin modificar la fachada 7

7.4. Inciso d) Instancia única, notificación de cambios y configuración 8

3 Pregunta 3

3.1 Inciso a) Identificación y justificación del patrón

El patrón más adecuado es **Visitor (Visitante)**, porque permite definir y añadir nuevas operaciones sobre una estructura de objetos (los elementos del documento) sin necesidad de modificar sus clases.

Se propone crear la interfaz `VisitanteDocumento`, que declara los métodos para visitar cada elemento concreto (`visitarParrafo()`, `visitarTabla()`, `visitarImagen()` y `visitarLista()`), y que será implementada por las clases de operaciones `ExportarHTML`, `ContarPalabras` y `ValidarAccesibilidad`. A su vez, los elementos (`Parrafo`, `Tabla`, `Imagen` y `Lista`) implementan la interfaz `ElementoDocumento` con el método `aceptar(visitante)`, el cual delega la ejecución a la operación correspondiente del visitante.

No se elige *Strategy* porque su objetivo es intercambiar diferentes algoritmos para una única operación dentro de un contexto, no aplicar múltiples operaciones distintas sobre una familia heterogénea de clases. Tampoco basta con polimorfismo simple, ya que obligaría a modificar todas las clases de elementos cada vez que se requiera agregar una nueva operación, incumpliendo el principio Abierto/Cerrado.

3.2 Inciso b) Implementación en pseudocódigo

```
INTERFAZ VisitanteDocumento
    MÉTODO visitarParrafo(parrafo)
    MÉTODO visitarTabla(tabla)
    MÉTODO visitarImagen(imagen)
    MÉTODO visitarLista(lista)
FIN INTERFAZ

INTERFAZ ElementoDocumento
    MÉTODO aceptar(visitante)
FIN INTERFAZ

CLASE Parrafo IMPLEMENTA ElementoDocumento
    MÉTODO aceptar(visitante)
        visitante.visitarParrafo(ESTE)
    FIN MÉTODO
FIN CLASE

CLASE Tabla IMPLEMENTA ElementoDocumento
    MÉTODO aceptar(visitante)
        visitante.visitarTabla(ESTE)
    FIN MÉTODO
FIN CLASE

CLASE Imagen IMPLEMENTA ElementoDocumento
    MÉTODO aceptar(visitante)
        visitante.visitarImagen(ESTE)
    FIN MÉTODO
FIN CLASE

CLASE Lista IMPLEMENTA ElementoDocumento
    MÉTODO aceptar(visitante)
        visitante.visitarLista(ESTE)
    FIN MÉTODO
```

```
FIN CLASE

CLASE ExportarHTML IMPLEMENTA VisitanteDocumento
    MÉTODO visitarParrafo(parrafo)
        MOSTRAR "<p>Párrafo exportado a HTML</p>"
    MÉTODO visitarTabla(tabla)
        MOSTRAR "<table>Tabla exportada a HTML</table>"
    MÉTODO visitarImagen(imagen)
        MOSTRAR "<img />Imagen exportada a HTML"
    MÉTODO visitarLista(lista)
        MOSTRAR "<ul>Lista exportada a HTML</ul>"
FIN CLASE

CLASE ContarPalabras IMPLEMENTA VisitanteDocumento
    MÉTODO visitarParrafo(parrafo)
        MOSTRAR "Contando palabras en Párrafo"
    MÉTODO visitarTabla(tabla)
        MOSTRAR "Contando palabras en celdas de Tabla"
    MÉTODO visitarImagen(imagen)
        MOSTRAR "Imagen no contiene palabras contabilizables"
    MÉTODO visitarLista(lista)
        MOSTRAR "Contando palabras en ítems de Lista"
FIN CLASE

CLASE ValidarAccesibilidad IMPLEMENTA VisitanteDocumento
    MÉTODO visitarParrafo(parrafo)
        MOSTRAR "Validando legibilidad del Párrafo"
    MÉTODO visitarTabla(tabla)
        MOSTRAR "Validando encabezados de la Tabla"
    MÉTODO visitarImagen(imagen)
        MOSTRAR "Validando texto alternativo (alt) de la Imagen"
    MÉTODO visitarLista(lista)
        MOSTRAR "Validando estructura semántica de la Lista"
FIN CLASE

ALGORITMO Principal
    CREAR parrafo COMO Parrafo
    CREAR tabla COMO Tabla
    CREAR imagen COMO Imagen
    CREAR lista COMO Lista

    CREAR exportador COMO ExportarHTML
    CREAR contador COMO ContarPalabras
    CREAR validador COMO ValidarAccesibilidad

    parrafo.aceptar(exportador)
    tabla.aceptar(contador)
    imagen.aceptar(validador)
    lista.aceptar(exportador)
FIN ALGORITMO
```

3.3 Inciso c) Documentos jerárquicos (Composite e Iterator)

Para manejar documentos jerárquicos (secciones que contienen subsecciones y elementos simples), se integra el patrón **Composite** mediante la clase `Seccion`, la cual también implementa `ElementoDocumento` y almacena una colección de elementos hijos. Para recorrer los elementos sin exponer la estructura interna de la sección, se integra el patrón **Iterator** a través de

IteradorElementos.

```
INTERFAZ Iterador
    MÉTODO tieneSiguiente()
    MÉTODO siguiente()
FIN INTERFAZ

CLASE IteradorElementos IMPLEMENTA Iterador
    elementos = []
    posicion = 0

    CONSTRUCTOR(listaElementos)
        ESTE.elementos = listaElementos
    FIN CONSTRUCTOR

    MÉTODO tieneSiguiente()
        RETORNAR posicion < LONGITUD(elementos)
    FIN MÉTODO

    MÉTODO siguiente()
        elemento = elementos[posicion]
        posicion = posicion + 1
        RETORNAR elemento
    FIN MÉTODO
FIN CLASE

CLASE Seccion IMPLEMENTA ElementoDocumento
    elementos = []

    MÉTODO agregar(elemento)
        AÑADIR elemento A elementos
    FIN MÉTODO

    MÉTODO crearIterador()
        RETORNAR NUEVO IteradorElementos(elementos)
    FIN MÉTODO

    MÉTODO aceptar(visitante)
        iterador = crearIterador()
        MIENTRAS iterador.tieneSiguiente()
            elemento = iterador.siguiente()
            elemento.aceptar(visitante)
        FIN MIENTRAS
    FIN MÉTODO
FIN CLASE

// Una Seccion puede contener tanto elementos simples (Parrafo, Tabla, etc.)
// como otras instancias de Seccion (subsecciones).
```

3.4 Inciso d) Comparativa y ejecución en paralelo

Comparación (Visitor vs. métodos en cada elemento):

- **Número de cambios por nueva operación:** Con *Visitor*, añadir una nueva operación requiere 0 modificaciones en las clases de elementos existentes (solo se crea una nueva clase visitante). Si se añaden métodos directamente en cada elemento, cada nueva operación exige modificar las N clases de elementos (4 cambios por operación en este caso).

- **Cuándo conviene cada uno:** *Visitor* conviene cuando la jerarquía de elementos es estable pero se agregan nuevas operaciones con frecuencia. Añadir métodos directamente en los elementos conviene cuando los tipos de elementos cambian o crecen constantemente y las operaciones son fijas.

Ejecución en paralelo y concurrencia: Si varios hilos comparten un mismo objeto *Visitor* con estado mutable (por ejemplo, un acumulador `totalPalabras`), se producirán condiciones de carrera (*race conditions*). Para ejecutar operaciones en paralelo sobre distintas secciones de forma segura, se instancia un *Visitor* independiente por cada hilo/sección y al finalizar se combinan los resultados parciales.

```
CLASE ContarPalabrasConcurrente IMPLEMENTA VisitanteDocumento
    totalPalabras = 0

    MÉTODO visitarParrafo(parrafo)
        totalPalabras = totalPalabras + 50
    FIN MÉTODO

    MÉTODO visitarTabla(tabla)
        totalPalabras = totalPalabras + 30
    FIN MÉTODO

    MÉTODO visitarImagen(imagen)
        totalPalabras = totalPalabras + 0
    FIN MÉTODO

    MÉTODO visitarLista(lista)
        totalPalabras = totalPalabras + 20
    FIN MÉTODO
FIN CLASE

ALGORITMO Principal
    seccion1 = NUEVA Seccion()
    seccion1.agregar(NUEVO Parrafo())

    seccion2 = NUEVA Seccion()
    seccion2.agregar(NUEVA Tabla())
    seccion2.agregar(NUEVA Lista())

    // Cada hilo utiliza su propia instancia de Visitor para evitar estado
       compartido mutable
    visitante1 = NUEVO ContarPalabrasConcurrente()
    visitante2 = NUEVO ContarPalabrasConcurrente()

    EJECUTAR_EN_PARALELO
        HILO 1: seccion1.aceptar(visitante1)
        HILO 2: seccion2.aceptar(visitante2)
    FIN EJECUTAR_EN_PARALELO

    totalDocumento = visitante1.totalPalabras + visitante2.totalPalabras
    MOSTRAR "Total de palabras: " + totalDocumento
FIN ALGORITMO
```

7 Pregunta 7

7.1 Inciso a) Identificación y justificación del patrón

El patrón más adecuado es **Facade (Fachada)**, porque proporciona una interfaz sencilla para controlar varios componentes complejos del sistema desde una única clase.

Se propone crear la clase `HomeTheaterFacade`, que coordine los seis componentes: `Amplificador`, `ReproductorDVD`, `ReproductorMusica`, `Proyector`, `Pantalla` y `Luces`. La fachada ofrecerá métodos como `verPelicula()`, `escucharMusica()` y `jugarVideojuego()`, encapsulando las secuencias necesarias para cada actividad.

No se elige *Mediator* porque su objetivo principal es centralizar la comunicación entre objetos que interactúan entre sí. Tampoco basta con un método estático, ya que no representa adecuadamente una fachada con dependencias, configuración y posibilidad de ampliación.

7.2 Inciso b) Implementación en pseudocódigo

```
CLASE Amplificador
    MÉTODO encender()
        MOSTRAR "Amplificador encendido"
    MÉTODO configurar(modo)
        MOSTRAR "Modo de sonido: " + modo
FIN CLASE

CLASE ReproductorDVD
    MÉTODO reproducir()
        MOSTRAR "Reproduciendo película"
FIN CLASE

CLASE ReproductorMusica
    MÉTODO reproducir()
        MOSTRAR "Reproduciendo música"
FIN CLASE

CLASE Proyector
    MÉTODO encender()
        MOSTRAR "Proyector encendido"
FIN CLASE

CLASE Pantalla
    MÉTODO encender()
        MOSTRAR "Pantalla encendida"
FIN CLASE

CLASE Luces
    MÉTODO atenuar()
        MOSTRAR "Luces atenuadas"
    MÉTODO encender()
        MOSTRAR "Luces encendidas"
FIN CLASE

CLASE HomeTheaterFacade
    CREAR amplificador COMO Amplificador
    CREAR dvd COMO ReproductorDVD
    CREAR musica COMO ReproductorMusica
    CREAR proyector COMO Proyector
```

```
    CREAR pantalla COMO Pantalla
    CREAR luces COMO Luces

    MÉTODO verPelicula()
        luces.atenuar()
        pantalla.encender()
        proyector.encender()
        amplificador.encender()
        amplificador.configurar("Cine")
        dvd.reproducir()
    FIN MÉTODO

    MÉTODO escucharMusica()
        luces.encender()
        amplificador.encender()
        amplificador.configurar("Estéreo")
        musica.reproducir()
    FIN MÉTODO

    MÉTODO jugarVideojuego()
        luces.encender()
        pantalla.encender()
        proyector.encender()
        amplificador.encender()
        amplificador.configurar("Videojuegos")
    FIN MÉTODO
FIN CLASE

ALGORITMO Principal
    CREAR home COMO HomeTheaterFacade

    home.verPelicula()
    home.escucharMusica()
    home.jugarVideojuego()
FIN ALGORITMO
```

7.3 Inciso c) Incorporación de nuevas actividades sin modificar la fachada

Para añadir nuevas actividades sin modificar la clase `HomeTheaterFacade`, se puede utilizar el patrón **Command**, encapsulando cada actividad en una clase independiente que implemente un método `ejecutar()`. Así, las nuevas actividades se agregan creando nuevos comandos, sin modificar la fachada. Otra alternativa es **Strategy**, si se busca intercambiar diferentes algoritmos o secuencias de funcionamiento.

```
INTERFAZ Comando
    MÉTODO ejecutar()
FIN INTERFAZ

CLASE ComandoVerPelicula IMPLEMENTA Comando
    MÉTODO ejecutar()
        home.verPelicula()
    FIN MÉTODO
FIN CLASE

CLASE ComandoEscucharMusica IMPLEMENTA Comando
    MÉTODO ejecutar()
        home.escucharMusica()
    FIN MÉTODO
```

```
FIN CLASE

CLASE ComandoNuevaActividad IMPLEMENTA Comando
    MÉTODO ejecutar()
        // Ejecutar la nueva secuencia de operaciones
    FIN MÉTODO
FIN CLASE

// Para añadir una actividad, se crea un nuevo comando.
// La fachada no necesita modificarse.
```

7.4 Inciso d) Instancia única, notificación de cambios y configuración

Se utiliza **Singleton** para garantizar que exista una única instancia de `HomeTheaterFacade` en el sistema. Además, se implementa **Observer** para notificar a los observadores cada vez que un componente cambia de estado, por ejemplo, cuando el amplificador se enciende o se apaga.

La configuración de las actividades puede almacenarse en un archivo YAML, que define las secuencias de operaciones de cada actividad.

```
CLASE HomeTheaterFacade
    instancia = NULO

    MÉTODO obtenerInstancia()
        SI instancia ES NULO
            instancia = NUEVA HomeTheaterFacade
        FIN SI
        RETORNAR instancia
    FIN MÉTODO
FIN CLASE

INTERFAZ Observador
    MÉTODO actualizar(componente, estado)
FIN INTERFAZ

CLASE Monitor IMPLEMENTA Observador
    MÉTODO actualizar(componente, estado)
        MOSTRAR componente + " cambió a " + estado
    FIN MÉTODO
FIN CLASE

CLASE Componente
    nombre
    estado = "Apagado"
    observadores = []

    CONSTRUCTOR(nombre)
        ESTE.nombre = nombre
    FIN CONSTRUCTOR

    MÉTODO agregarObservador(observador)
        AÑADIR observador A observadores
    FIN MÉTODO

    MÉTODO cambiarEstado(nuevoEstado)
        estado = nuevoEstado

        PARA CADA observador EN observadores
```

```
            observador.actualizar(nombre, estado)
        FIN PARA
    FIN MÉTODO
FIN CLASE

CLASE Amplificador HEREDA Componente
    CONSTRUCTOR()
        nombre = "Amplificador"
        estado = "Apagado"
        observadores = []
    FIN CONSTRUCTOR
FIN CLASE

ALGORITMO Principal
    home = HomeTheaterFacade.obtenerInstancia()

    amplificador = NUEVO Amplificador()
    monitor = NUEVO Monitor()

    amplificador.agregarObservador(monitor)
    amplificador.cambiarEstado("Encendido")
FIN ALGORITMO
```